

```
Python
import datetime
```

```
md_content = """"# Enterprise IT Asset Management System (Project Tracer)
## Document 3: High-Level Design (HLD)
```

```
**Prepared By:** Sakthivel P, Principal Enterprise Architect
**Document Version:** 1.0
**Status:** Approved for Architectural Review
```

```
---
```

1. Introduction

This High-Level Design (HLD) document outlines the architectural blueprints for Project Tracer. Utilizing the C4 model (Context, Container, Component), alongside detailed sequence and deployment diagrams, this document provides a comprehensive visual and narrative guide to how the system is structured, deployed, and operates at scale using ASP.NET Core 9, Angular 20+, and SQL Server.

```
---
```

2. C4 Architecture Models

2.1 Level 1: System Context Diagram

This diagram illustrates Tracer within the broader enterprise ecosystem, highlighting interactions with external actors and systems.

Code output

Document 3 successfully generated: Tracer_HLD_Document3.md

```
mermaid
graph TD
    User((End User / IT Staff))
    Admin((System Admin))
```



Tracer_HLD_Document3

MD
Open

```
subgraph Enterprise Ecosystem
  Tracer[Tracer ITAM System]
  IdP[Azure AD / OIDC Provider]
  SMTP[SMTP / Email Gateway]
  Messaging[Slack / Teams Webhooks]
  ERP[ERP / Procurement System]
end
```

User -->|Interacts via Browser/Mobile| Tracer
Admin -->|Configures & Manages| Tracer

Tracer -->|Authenticates & Syncs SCIM| IdP
Tracer -->|Sends Notifications| SMTP
Tracer -->|Dispatches Alerts| Messaging
ERP -->|Pushes Procurement Data via API| Tracer

style Tracer fill:#bbf,stroke:#333,stroke-width:4px

****Component Explanation:****

*** **Tracer ITAM System:**** The core application boundaries.

*** **Azure AD / IdP:**** Provides centralized identity management and SCIM-based automated user provisioning.

*** **SMTP & Messaging:**** External communication channels for automated alerts (e.g., checkout EULAs, low stock warnings).

*** **ERP:**** Source of truth for initial hardware purchasing data, feeding into Tracer's pending inventory via secure REST APIs.

2.2 Level 2: Container Diagram

Breaks down the Tracer system into its major executing containers.

```
```mermaid
```

```
graph TD
```

```
 User((User)) -->|HTTPS| SPA[Angular 20+ SPA]
```

SPA -->|REST / SignalR| Gateway[API Gateway / YARP]

Gateway -->|Routes Requests| API[ASP.NET Core 9 API]

Gateway -->|Routes Requests| Background[Background Worker Service]

API -->|Read/Write| DB[(SQL Server Database)]

API -->|Cache Check| Cache[(Redis Cache)]

API -->|Blob Storage| Storage[(S3 / Azure Blob Storage)]

Background -->|Batch Processing| DB

Background -->|Dispatch Emails| SMTP[External SMTP]

### Component Explanation:

- **Angular 20+ SPA:** The static frontend application, utilizing Angular Signals for reactive state.
- **API Gateway (YARP):** Acts as the reverse proxy, handling SSL termination, rate limiting, and routing.
- **ASP.NET Core 9 API:** The core monolithic application hosting the Clean Architecture backend.
- **Background Worker Service:** Dedicated .NET Generic Host for executing cron jobs (e.g., license expiration checks, daily backups).
- **SQL Server:** Relational storage utilizing Temporal Tables for the Activity Log.
- **Redis Cache:** Distributed cache for session state, API response caching, and MediatR query caching.
- **S3/Blob Storage:** Unstructured storage for asset photos, PDF manuals, and EULA signatures.

## 2.3 Level 3: Component Diagram (Backend API)

Details the internal structure of the ASP.NET Core API container utilizing Clean Architecture and CQRS.

Code snippet

graph TD

subgraph Presentation

    Controllers[Minimal APIs / Endpoints]

end

subgraph Application Layer

    MediatR[MediatR Bus]

    Commands[Command Handlers]

    Queries[Query Handlers]

```
Validation[FluentValidation]
end
```

```
subgraph Domain Layer
 Entities[Aggregates & Entities]
 Rules[Domain Services & Rules]
end
```

```
subgraph Infrastructure Layer
 EFCore[EF Core DbContext]
 Repos[Repository Implementations]
 EmailService[Email Provider]
 StorageService[Blob Storage Provider]
end
```

```
Controllers -->|Dispatches| MediatR
MediatR -->|Pipelines via| Validation
MediatR --> Commands
MediatR --> Queries
```

```
Commands --> Entities
Commands --> Repos
Queries --> Repos
```

```
Repos --> EFCore
```

#### Component Explanation:

- **Minimal APIs:** Thin routing layer that strictly accepts HTTP requests and pushes them onto the MediatR bus.
- **MediatR Bus:** Implements the CQRS pattern. Write operations (Commands) and Read operations (Queries) are strictly separated.
- **FluentValidation:** Pipeline behavior that intercepts MediatR requests and rejects invalid payloads before they reach handlers.
- **Domain Entities:** The rich domain model (e.g., Asset, License) containing business logic and invariants. Zero external dependencies.
- **Infrastructure (EF Core):** Concrete implementation of database access, translating domain interactions into SQL queries.

## 3. Deployment Architecture

### 3.1 Kubernetes Deployment Diagram

Tracer is designed for containerized deployment using Docker and Kubernetes.

Code snippet

graph TD

```
subgraph Cloud_Provider_Region [Cloud Provider Region]
 LB[Load Balancer]

 subgraph Kubernetes_Cluster [Kubernetes Cluster]
 subgraph Tracer_Namespace [Tracer Namespace]
 UI_Pods[Angular UI Pods x3]
 API_Pods[API Pods x3]
 Worker_Pods[Worker Pods x2]
 end
 end

 subgraph Managed_Services [Managed Services]
 SQL_Managed[(Managed SQL Server)]
 Redis_Managed[(Managed Redis)]
 Blob_Managed[(Blob Storage Account)]
 end
end

Client((Clients)) -->|HTTPS| LB
LB --> Ingress
Ingress -->|Static Assets| UI_Pods
Ingress -->|/api/*| API_Pods

API_Pods --> SQL_Managed
API_Pods --> Redis_Managed
API_Pods --> Blob_Managed

Worker_Pods --> SQL_Managed
Worker_Pods --> SMTP((External SMTP))
```

#### Component Explanation:

- **Horizontal Pod Autoscaling (HPA):** API and UI pods scale dynamically based on CPU/Memory thresholds.
- **Managed Services:** Database, Caching, and Storage are offloaded to cloud-native PaaS offerings for high availability and automated backups.

## 4. Sequence Diagrams & Workflows

### 4.1 Authentication & Authorization Flow (OIDC)

Code snippet

sequenceDiagram

actor User

participant SPA as Angular UI

participant IdP as Azure AD (OIDC)

participant API as Tracer API (Auth Middleware)

User->>SPA: Click 'Login'

SPA->>IdP: Redirect to IdP Login Page

User->>IdP: Enter Credentials & MFA

IdP-->>SPA: Return Auth Code

SPA->>API: Exchange Code for Access/Refresh Tokens

API->>IdP: Validate Code

IdP-->>API: Tokens Validated

API->>API: Map Claims to Internal Roles

API-->>SPA: Return JWT & User Profile

SPA->>SPA: Store JWT securely

User->>SPA: Access Protected Route

SPA->>API: HTTP Request + Bearer JWT

API->>API: Validate Token & RBAC Claims

API-->>SPA: Return 200 OK / Data

### 4.2 Asset Checkout Workflow

Code snippet

sequenceDiagram

actor Tech as IT Technician

participant API as API Controller

participant MediatR as MediatR Pipeline

participant DB as SQL Server

participant Blob as Storage (EULA)

```
Tech->>API: POST /api/assets/checkout {AssetID, UserID}
API->>MediatR: Dispatch CheckoutAssetCommand
MediatR->>MediatR: Validate User & Asset Exists
MediatR->>DB: Check Asset Status (Must be 'Deployable')
```

```
alt Status invalid or already assigned
 DB-->>MediatR: Invalid State
 MediatR-->>API: 422 Unprocessable Entity
 API-->>Tech: Error Message
else Valid for Checkout
 MediatR->>DB: Update Asset Target & Status
 MediatR->>DB: Insert Activity Log (Temporal)
 MediatR->>Blob: Generate & Store EULA PDF
 DB-->>MediatR: Transaction Committed
 MediatR-->>API: Success Response + EULA Link
 API-->>Tech: 200 OK
end
```

### 4.3 License True-Up & Allocation Workflow

Code snippet

sequenceDiagram

```
participant Worker as Background Job
participant DB as SQL Server
participant Email as Notification Engine
```

```
Worker->>Worker: Trigger Daily License Sync
Worker->>DB: Query Licenses where (Allocated > TotalSeats)
DB-->>Worker: Return Over-allocated License List
```

```
alt No Violations
 Worker->>Worker: Exit Job
else Violations Found
 Worker->>DB: Query IT Admin Contact List
 Worker->>Email: Dispatch True-Up Alert Email (HTML)
 Worker->>DB: Log System Alert Event
end
```

### 4.4 Data Import Pipeline (CSV)

Code snippet

sequenceDiagram

actor Admin

participant UI as Angular SPA

participant API as Import Controller

participant Parser as CSV Validation Engine

participant DB as SQL Server

Admin->>UI: Upload assets.csv

UI->>API: POST /api/import (multipart/form-data)

API->>Parser: Parse & Validate Rows

alt Format Errors Found

Parser-->>API: Return Validation Error Array (Row, Col)

API-->>UI: 400 Bad Request + Error UI Map

else Validation Passed

Parser->>DB: Bulk Insert via SqlBulkCopy (Staging Table)

DB->>DB: Execute MERGE into Assets Table

DB-->>API: Import Complete (Rows Affected)

API-->>UI: 200 OK

end

## 5. Sub-System Architectures

### 5.1 Logging & Activity Architecture

Tracer utilizes two distinct logging paradigms:

1. **Application Telemetry (Serilog + OpenTelemetry):** Captures HTTP requests, exceptions, and performance metrics. Routed to ELK (Elasticsearch, Logstash, Kibana) or Datadog.
2. **Domain Activity Logging (SQL Temporal Tables):** Business-level audit trails. Every domain entity (Asset, License, User) maps to a SQL Temporal Table. Any UPDATE or DELETE automatically pushes the previous state row into an immutable History table, linked by the transaction timestamp and User ID.

### 5.2 Caching Strategy

- **L1 Cache (In-Memory):** Scoped to the MediatR request pipeline to prevent duplicate database queries within a single HTTP request boundary.
- **L2 Cache (Redis Distributed):** Used for highly concurrent read models.



- *Example:* The Categories and Locations drop-down lists are cached in Redis. When an Admin adds a new Category, an EntityCreatedEvent invalidates the specific Redis key.

## 5.3 Background Jobs Framework

Implemented using **Quartz.NET** integrated into the ASP.NET Core Generic Host.

- **Job Persistence:** Quartz utilizes SQL Server to persist job triggers. If a worker pod crashes, another pod will pick up the stalled job.
- **Key Scheduled Jobs:**
  - DailyDepreciationCalculatorJob: Updates financial current-value fields.
  - LicenseExpirationNotifierJob: Scans for licenses expiring in 30/15/7 days.
  - AdSyncWorker: Polls Azure AD for delta changes if SCIM push is not utilized.

*End of Document 3. Awaiting next instruction.*

""

```
with open("Tracer_HLD_Document3.md", "w", encoding="utf-8") as f:
```

```
f.write(md_content)
```

```
print("Document 3 successfully generated: Tracer_HLD_Document3.md")
```

Your Markdown file is ready

The High-Level Design (Document 3) has been fully generated.

It utilizes the requested C4 architectural model (Context, Container, and Component diagrams) alongside detailed sequence diagrams mapping out authentication, asset checkout, license workflows, and import pipelines. All diagrams use Mermaid syntax. Furthermore, sub-system architectures like Caching, Logging, and Background Jobs are explicitly detailed according to the ASP.NET Core and Angular specifications.

Awaiting your next instruction.